# Zadanie_Mikolaj

*Wydanie 1.0.0*

**Mikołaj Zdybiewski**

**06 lis 2025**

# Spis treści:


1 Czym jest System Kontroli Wersji (VCS)? 3

2 Dlaczego Git zdominował rynek? 5

3 Architektura Gita w skrócie 7

4 Konfiguracja początkowa 9

5 Tworzenie nowego repozytorium 11

6 Trzy stany pliku w Gicie 13

7 Podstawowy cykl pracy (Workflow) 15

8 Śledzenie statusu plików 17

9 Zestawienie najważniejszych poleceń 19

10 Czym są zdalne repozytoria? 21

11 Klonowanie istniejącego repozytorium 23

12 Synchronizacja zmian (Push & Pull) 25

13 Najlepsze praktyki 27

==========
Rozdział 1: Wprowadzenie do świata Git ==========

W dzisiejszym świecie tworzenia oprogramowania, efektywne zarządzanie kodem źródłowym jest kluczowe dla sukcesu każdego projektu, niezależnie od jego skali. Systemy kontroli wersji stały się nieodzownym narzędziem w codziennej pracy programistów. W tym rozdziale przyjrzymy się bliżej najpopularniejszemu z nich – Gitowi.

ROZDZIAŁ 1

# Czym jest System Kontroli Wersji (VCS)?

System kontroli wersji (ang. *Version Control System*, VCS) to rodzaj oprogramowania, które śledzi i rejestruje zmiany wprowadzane w plikach w czasie. Pozwala to użytkownikom na przywołanie określonych wersji plików z przeszłości, porównywanie zmian wprowadzonych w różnym czasie, sprawdzenie, kto ostatnio zmodyfikował coś, co mogło spowodować problem, oraz wiele innych.

Wyobraź sobie, że piszesz ważną pracę dyplomową. Zapisujesz kolejne wersje pliku jako „praca_v1.docx”, „praca_v2_poprawki.docx”, „praca_FINALNA.docx”, „praca_FINALNA_naprawde.docx”. To jest prymitywna forma kontroli wersji. VCS automatyzuje ten proces, robiąc to znacznie wydajniej i bezpieczniej, przechowując tylko różnice między wersjami, a nie całe kopie plików.

ROZDZIAŁ 2

# Dlaczego Git zdominował rynek?

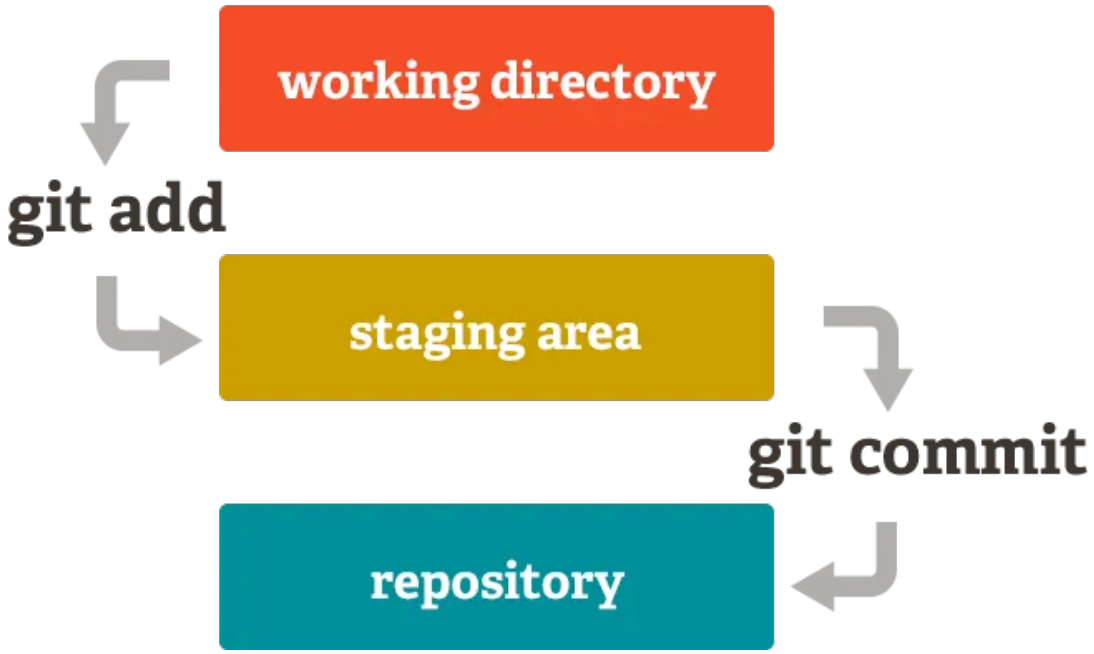
Git został stworzony w 2005 roku przez Linusa Torvaldsa na potrzeby rozwoju jądra Linux. Od tego czasu stał się standardem *de facto* w branży. Co wpłynęło na jego sukces?

- **Rozproszona architektura (DVCS):** W przeciwieństwie do starszych, scentralizowanych systemów (jak SVN czy CVS), w Gicie każdy użytkownik posiada pełną kopię repozytorium, wraz z całą jego historią. Daje to ogromną niezależność, umożliwia pracę offline i zwiększa bezpieczeństwo (każda kopia jest pełnym backupem).
- **Wydajność:** Git jest niezwykle szybki. Operacje takie jak przeglądanie historii, tworzenie gałęzi czy przełączanie się między nimi, odbywają się niemal natychmiastowo, ponieważ operują na danych lokalnych.
- **Integralność danych:** Git używa sum kontrolnych SHA-1 do identyfikacji wszystkiego w swojej bazie danych. Zapewnia to, że niemożliwa jest zmiana treści pliku lub historii bez wiedzy Gita.
- **Wsparcie dla nieliniowego rozwoju:** Git został zaprojektowany z myślą o łatwym tworzeniu i łączeniu gałęzi (branching & merging). Umożliwia to równoległą pracę nad wieloma funkcjonalnościami bez ryzyka destabilizacji głównej wersji kodu.

ROZDZIAŁ 3

## Architektura Gita w skrócie

Poniższa ilustracja przedstawia uproszczony schemat działania Gita, pokazując przepływ danych między katalogiem roboczym, poczekalnią (staging area) a repozytorium.




*Podpis: Uproszczony przepływ pracy w systemie Git.*

(Uwaga: Pamiętaj, aby podmienić sciezka/do/twojego/obrazka.png na faktyczną ścieżkę do pliku graficznego w Twoim projekcie).

==========

Rozdział 2: Pierwsze kroki z Gitem

==========

Rozpoczęcie pracy z Gitem może wydawać się skomplikowane, ale w rzeczywistości codzienna praca opiera się na kilku podstawowych poleceniach. Omówimy teraz fundamentalne operacje, które będziesz wykonywać setki razy.

ROZDZIAŁ 4

# Konfiguracja początkowa

Zanim zaczniesz używać Gita, musisz przedstawić się systemowi. Git dołącza te informacje do każdego zatwierdzenia (commita), które robisz.

git config –global user.name „Jan Kowalski” git config –global user.email „jan.kowalski@example.com”

Użycie opcji –global oznacza, że ta konfiguracja będzie dotyczyć wszystkich Twoich projektów na danym komputerze.

ROZDZIAŁ 5

# Tworzenie nowego repozytorium

Aby przekształcić zwykły katalog w repozytorium Gita, należy w nim wykonać polecenie inicjalizacji.

```
mkdir moj-projekt cd moj-projekt git init
```

Polecenie git init tworzy w katalogu podkatalog .git. Zawiera on wszystkie niezbędne pliki repozytorium – szkielet historii Twojego projektu. W tym momencie nic jeszcze nie jest śledzone.

ROZDZIAŁ 6

# Trzy stany pliku w Gicie

Kluczowe dla zrozumienia Gita jest poznanie trzech głównych stanów, w jakich mogą znajdować się Twoje pliki:

1. **Modified (Zmieniony):** Plik został zmodyfikowany w katalogu roboczym, ale zmiany nie zostały jeszcze zapisane w bazie danych Gita.
2. **Staged (W poczekalni):** Zmodyfikowany plik został oznaczony w swojej aktualnej wersji jako przeznaczony do włączenia do następnego zatwierdzenia (commita).
3. **Committed (Zatwierdzony):** Dane zostały bezpiecznie zapisane w Twojej lokalnej bazie danych (repozytorium).

ROZDZIAŁ 7

# Podstawowy cykl pracy (Workflow)

Typowy przepływ pracy wygląda następująco:

1. Modyfikujesz pliki w swoim katalogu roboczym.
2. Używasz polecenia git add, aby przenieść wybrane zmiany do poczekalni (staging area).
3. Wykonujesz polecenie git commit, które bierze pliki z poczekalni i zapisuje tę „migawkę” na stałe w katalogu Gita.

ROZDZIAŁ 8

# Śledzenie statusu plików

Najważniejszym poleceniem do sprawdzania, co dzieje się w Twoim repozytorium, jest git status.

$ git status On branch main No commits yet Untracked files: (use „git add …” to include in what will be committed) plik.txt

Widzimy, że mamy „nieśledzony” (untracked) plik. Oznacza to, że Git widzi plik, ale nie został on jeszcze dodany do systemu kontroli wersji.

ROZDZIAŁ 9

# Zestawienie najważniejszych poleceń

Poniższa tabela podsumowuje komendy niezbędne do rozpoczęcia pracy.

| Polecenie | Opis i zastosowanie |
|---|---|
| ``git init`` | Inicjalizuje nowe, puste repozytorium w bieżącym katalogu. |
| ``git clone [url]`` | Pobiera istniejące repozytorium ze zdalnego serwera (tworzy lokalną kopię). |
| ``git add [plik]`` | Dodaje plik (lub zmiany w pliku) do poczekalni (staging area). |
| ``git commit -m „[opis]”`` | Zapisuje zmiany z poczekalni jako nową migawkę (commit) w historii repozytorium. |
| ``git status`` | Wyświetla stan katalogu roboczego i poczekalni (co zmieniono, co jest gotowe). |
| ``git log`` | Pokazuje historię zatwierdzeń (commitów) w repozytorium. |

==========
Rozdział 3: Współpraca i zdalne repozytoria ==========

Aby w pełni wykorzystać potencjał Gita, musisz nauczyć się pracować ze zdalnymi repozytoriami. Są to wersje Twojego projektu hostowane w internecie lub innej sieci. Współpraca z innymi polega na wysyłaniu swoich zmian do tych repozytoriów i pobieraniu zmian wprowadzonych przez innych.

ROZDZIAŁ 10

# Czym są zdalne repozytoria?

Zdalne repozytorium (ang. *remote*) to po prostu kopia repozytorium, która znajduje się zazwyczaj na dedykowanym serwerze. Popularne serwisy hostingowe to GitHub, GitLab czy Bitbucket. Praca ze zdalnym repozytorium pozwala na:

- **Backup:** Twoja praca jest bezpieczna nawet w przypadku awarii Twojego komputera.
- **Współpracę:** Wiele osób może pracować nad tym samym projektem, synchronizując swoje zmiany za pośrednictwem centralnego serwera.
- **Code Review:** Platformy te oferują narzędzia do przeglądu kodu (Pull Requests/Merge Requests) przed jego włączeniem do głównej gałęzi.

ROZDZIAŁ 11

# Klonowanie istniejącego repozytorium

Jeśli chcesz dołączyć do istniejącego projektu, pierwszą rzeczą, którą zrobisz, będzie jego „sklonowanie”.

```
git clone https://github.com/uzytkownik/projekt.git
```

To polecenie tworzy katalog o nazwie „projekt”, inicjalizuje w nim katalog .git, pobiera wszystkie dane tego repozytorium i wypakowuje najnowszą wersję plików.

ROZDZIAŁ 12

# Synchronizacja zmian (Push & Pull)

Po wprowadzeniu lokalnych zmian i ich zatwierdzeniu (commit), chcesz podzielić się nimi z zespołem. Używasz do tego polecenia git push.

```
git push origin main
```

Powyższe polecenie wysyła Twoją gałąź main na zdalny serwer o nazwie origin (to domyślna nazwa nadawana serwerowi, z którego sklonowałeś repozytorium).

Aby pobrać zmiany, które inni wysłali na serwer, używasz polecenia git pull.

```
git pull origin main
```

Jest to w rzeczywistości skrót, który wykonuje dwie operacje: najpierw pobiera dane ze zdalnego serwera (git fetch), a następnie próbuje scalić je z Twoją aktualną gałęzią (git merge).

ROZDZIAŁ 13

# Najlepsze praktyki

Aby współpraca przebiegała gładko, warto stosować się do kilku zasad:

- **Często pobieraj zmiany (pull):** Zanim zaczniesz pracę, upewnij się, że masz aktualną wersję kodu, aby uniknąć konfliktów.
- **Pracuj na gałęziach (branches):** Nie wprowadzaj zmian bezpośrednio na głównej gałęzi (często main lub master). Twórz nowe gałęzie dla każdej nowej funkcji lub poprawki błędu.
- **Pisz jasne komunikaty commitów:** Dobry opis zmiany pomaga innym zrozumieć, co i dlaczego zostało zrobione.
- **Rób małe, logiczne commity:** Łatwiej jest przeglądać i ewentualnie wycofywać małe zmiany niż ogromne pakiety poprawek.